Exact Equilibrium Solutions for the Board Game *Football Strategy*: A Full-Game Backward-Induction Solver

An Extension of McCulloch's Game-Theoretic Agent

George Kyrollos

Abstract

McCulloch [1] demonstrated that each individual scrimmage play in Avalon Hill's board game *Football Strategy* constitutes a two-player zero-sum matrix game, and designed an agent that solves each play in isolation using mixed-strategy Nash equilibrium. To make the agent sensitive to game context—field position, down, distance, score, and clock—McCulloch augmented the raw yardage payoffs with hand-crafted heuristic bonuses. We replace that heuristic layer entirely. By modeling the complete game as a finite-horizon two-player zero-sum stochastic game and applying exact backward induction, every matrix-game entry becomes the true equilibrium continuation value of the resulting game state. The solver determines optimal play across all ∼30 million reachable states of the fourth quarter, selecting between punt, field goal, and scrimmage plays based on actual win probability rather than proxy rewards. Each team's three timeouts per half are modeled as a pure-strategy sequential sub-game resolved after each play, so the solution reflects optimal timeout usage. We describe the mathematical formulation, state-space representation, solution algorithm, and computational architecture, and compare the approach explicitly to McCulloch's method.

1 Introduction

Football Strategy (Avalon Hill, 1969) is a two-player board game that abstracts American football into a sequence of simultaneous-move interactions. On each scrimmage play the offense secretly selects one of up to 22 actions while the defense simultaneously selects one of ten cards (A–J); the intersection on a printed chart determines the result—yardage, an incomplete pass, a fumble, an interception, a penalty, or a score. Because both players choose without observing the opponent's action, the play is exactly a $|O| \times 10$ two-player zero-sum matrix game.

McCulloch [1] was the first to exploit this structure algorithmically. His agent solves each matrix game using linear programming to obtain a mixed-strategy Nash equilibrium. The key challenge is assigning *utility* to each chart outcome: a raw yardage gain of +7 is not inherently better or worse than a gain of +4 without knowing the down, distance, score, clock, and field position. McCulloch addressed this with a set of hand-tuned bonuses—for first downs, touchdowns, turnovers, and fourth-down conversions—that effectively map game states to a scalar reward. While principled in spirit, these heuristics are necessarily approximate and do not guarantee that the agent's play is optimal for the actual game of *Football Strategy*.

Our contribution is to eliminate the heuristic layer completely. Instead of assigning an ad-hoc scalar to each outcome, we assign it the *exact equilibrium continuation value* of the full game state it produces. This requires solving ∼30 million linked matrix games simultaneously via backward

induction over the complete finite game tree. The result is a provably optimal strategy—in the sense of the minimax theorem for finite two-player zero-sum stochastic games—for the fourth quarter of *Football Strategy*.

2 McCulloch's Approach and Its Limitations

McCulloch's agent operates as follows. At each scrimmage state it constructs a payoff matrix $A \in \mathbb{R}^{|O| \times 10}$ where entry $A_{a,b}$ is the utility assigned to the outcome produced by offensive action a and defensive card b. The matrix game $\max_{\sigma_O} \min_{\sigma_D} \sigma_O^\top A \, \sigma_D$ is then solved via LP to obtain mixed strategies.

The utility function $A_{a,b}$ is where the approximation lives. McCulloch converts each chart result into a scalar by applying additive bonuses:

- A first down earns a bonus proportional to the fraction of the field gained.
- A touchdown earns a large positive bonus scaled by the score context.
- An interception or fumble lost earns a large negative bonus.
- A fourth-down conversion earns a bonus; failing on fourth down earns a penalty.
- Clock and score interact only through the bonus magnitudes, which are set by hand.

The limitations of this approach are fundamental rather than incidental:

Heuristics cannot capture non-monotone value. The value of gaining 10 yards on first-and-10 at midfield in the final minute while trailing by 3 is qualitatively different from the value of the same play with ten minutes remaining. No fixed additive bonus captures this interaction without effectively re-solving the game.

Optimal fourth-down decisions require global knowledge. Whether to punt, attempt a field goal, or go for it on fourth down depends on score, clock, and field position in ways that require comparing continuation values under each option. McCulloch's agent applies fixed cutoffs; the correct decision is state-dependent and only determinable via backward induction.

End-of-half clock management is inaccessible. The strategic value of running out the clock, spiking the ball, or calling a timeout is zero under heuristics but central to real play. Capturing it requires the value function to be defined over clock states.

3 Formal Game Model

3.1 Terminal Utility

We model *Football Strategy* as a finite-horizon two-player zero-sum stochastic game. Let $\delta \in \mathbb{Z}$ denote the score differential (Team 1 minus Team 2) at the end of the game. The terminal utility is

$$U(\delta) = \operatorname{sgn}(\delta) \in \{-1, 0, +1\},$$

so the objective is to maximize win probability, not expected point margin. This is the correct objective for a game whose outcome is win, loss, or draw.

3.2 State Representation

A complete game state must encode every quantity that affects future optimal play. We use a *symmetric possession* convention: the state is always written from the perspective of the team currently in possession of the ball, so field position $x \in [1, 99]$ is always measured in yards from the current offense's own goal line ($x = 0$: own goal; $x = 100$: opponent's goal).

Definition 1 (Scrimmage state)**.** *A scrimmage state is a tuple*

$$s = (q,\, \tau,\, x,\, d,\, y,\, \delta,\, h,\, \theta_1,\, \theta_2),$$

where:

- $q \in \{1, 2, 3, 4\}$ *is the quarter;*
- $\tau \in \{0, 1, \ldots, 60\}$ *is the number of 15-second ticks remaining in the quarter;*
- $x \in \{1, \ldots, 99\}$ *is ball position (yards from current offense's own goal);*
- $d \in \{1, 2, 3, 4\}$ *is the current down;*
- $y \in \{1, \ldots, 30\}$ *is yards to first down (capped at 30);*
- $\delta \in \{-35, \ldots, 35\}$ *is the score differential from the current offense's perspective (clipped at* ± 35*);*
- $h \in \{-1, 0, +1\}$ *encodes which team receives the second-half kickoff (*$h = 0$ *in Q3/Q4 when this is already resolved); and*
- $\theta_1, \theta_2 \in \{0, 1, 2, 3\}$ *are the remaining timeouts for the current offense and defense, respectively.*

The full game starts at $\theta_1 = \theta_2 = 3$.
In addition to scrimmage states, the game includes *kickoff states* (q, τ, δ, h) and *safety-kick states* (q, τ, δ, h).

Possession symmetry. Because the state is always expressed from the current offense's viewpoint, a possession change transforms the state as

$$x \;\mapsto\; 100 - x, \qquad \delta \;\mapsto\; -\delta, \qquad h \;\mapsto\; -h,$$

and the continuation value is negated (since the current maximizer becomes the minimizer). This halves the effective state space compared to a possession-explicit encoding.

3.3 Transition Kernel

The transition kernel $P(s' \mid s, a, b)$ is determined by:

1. **Chart lookup:** the cell at row b (defense card), column a (offense play) of the selected offense chart.

2. **Outcome parsing:** chart cells encode one of: a deterministic yardage gain or loss (possibly out-of-bounds), an incomplete pass, a fumble, an interception with return yards, a long gain (LG), a penalty, a loss-of-down, or an OR-choice between two outcomes resolved by the favored team.

3. **Game-rule application:** yardage is added to x; if $x \geq 100$ a touchdown results; if $x \leq 0$ a safety results; first downs reset d and y; fourth-down failures transfer possession.

4. **Clock advancement:** τ is decremented by the tick cost of the outcome type (Table 1), subject to the two-minute automatic stoppage at $\tau = 8$ in Q2 and Q4.

Table 1: Time consumed per outcome category (15 sec per tick).

Outcome type	Time (sec)	Ticks
In-bounds gain of 0–19 yards	30	2
In-bounds gain of ≥ 20 yards	45	3
Any loss	30	2
Out-of-bounds play	15	1
Incomplete pass	15	1
Interception	30	2
Fumble	15	1
Penalty	15	1
Kickoff / field goal / punt	15	1
Extra point (PAT)	0	0

Long Gain. Chart cells marked `LG` trigger a probabilistic outcome drawn from the distribution in Table 2, which was derived from the printed Long Gain table. LG outcomes always consume 3 ticks.

Table 2: Long Gain distribution.

Gain (yards)	Probability
+30, +35, +40, +45, +50	$\frac{1}{6}$ each
+60, +70, …, +110	$\frac{1}{36}$ each

Field goals. A field goal attempt is legal on any down when $x \geq 55$ (within 45 yards of the opponent's goal line). Success probabilities are distance-based (Table 3).

Extra points. After every touchdown the scoring team attempts a PAT with $P(\text{good}) = \frac{34}{36}$. PATs consume zero clock time.

Table 3: Field goal success probabilities by distance.

Distance to goal line (yards)	$P(\text{good})$
1–12	$\frac{32}{36}$
13–22	$\frac{28}{36}$
23–32	$\frac{18}{36}$
33–38	$\frac{12}{36}$
39–45	$\frac{2}{36}$

4 Solution Algorithm

4.1 The Bellman–Minimax Recursion

Let $V(s)$ denote the equilibrium value of state s from the perspective of the current offense (Team 1 when Team 1 has the ball, Team 2's negative when Team 2 has the ball). The terminal condition is $V(q = 4, \tau = 0, \cdot) = U(\delta)$.

For each scrimmage state s, let $O(s) \subseteq \{1, \ldots, 20\} \cup \{\textsc{punt}, \textsc{fg}\}$ be the set of legal offensive actions and $D = \{A, \ldots, J\}$ the set of defensive cards. Define the expected continuation value of the action pair (a, b):

$$Q_s(a, b) = \sum_{s'} P(s' \mid s, a, b)\, V(s'),$$

where the sum is over successor states. The equilibrium value satisfies

$$V(s) = \max_{\sigma_O \in \Delta(O(s))} \min_{\sigma_D \in \Delta(D)} \sum_{a \in O(s)} \sum_{b \in D} \sigma_O(a)\, \sigma_D(b)\, Q_s(a, b). \tag{1}$$

By the minimax theorem, the max and min can be exchanged, and the common value equals $V(s)$.

4.2 Linear Programming Formulation

For each scrimmage state s, we assemble the payoff matrix $A^{(s)} \in \mathbb{R}^{|O(s)| \times 10}$ with $A^{(s)}_{a,b} = Q_s(a, b)$ and solve the row player's (offense's) LP:

$$\begin{aligned} \text{maximize} \quad & v \\ \text{subject to} \quad & \sum_a p_a\, A^{(s)}_{a,b} \geq v \quad \forall b \in D, \\ & \sum_a p_a = 1, \quad p_a \geq 0 \quad \forall a \in O(s). \end{aligned}$$

The optimal value $v^* = V(s)$ and the optimal p^* is the equilibrium mixed strategy for the offense. An analogous LP yields the defense's equilibrium mixed strategy. Both LPs are solved using the HiGHS interior-point and dual-simplex solvers via `scipy.optimize.linprog`.

4.3 Backward Induction and Forward Reachability

Because states at clock time τ depend only on states at $\tau' < \tau$ (the game moves forward in real time), solving from $\tau = 1$ upward within each quarter, then from $Q4$ backward to $Q1$, gives a correct backward-induction order.

Algorithm 1 summarizes the procedure. The forward BFS ensures that only reachable states are solved, which reduces the state count substantially.

Algorithm 1 Full-game backward induction solver

Require: Offense chart C, punt chart P, start state s_0
$\mathcal{S} \leftarrow \textsc{ForwardBFS}(C, P, s_0)$ ▷ Enumerate all reachable states
for $q \in \{4, 3, 2, 1\}$ **do**
  Apply quarter-boundary seeding (e.g. halftime kickoff)
  for $\tau \in \{1, 2, \ldots, 60\}$ **do**
    for each reachable state s at (q, τ) **in parallel do**
      if s is a scrimmage state **then**
        Build $A^{(s)}$ by calling transition engine for each $(a, b) \in O(s) \times D$
        $V(s) \leftarrow$ LP value of $A^{(s)}$
      else if s is a kickoff or safety-kick state **then**
        $V(s) \leftarrow$ closed-form expected value over kickoff/punt table
      end if
    end for
  end for
end forreturn V

Each tau layer is embarrassingly parallel: states at (q, τ) look up values only at already-solved (q, τ') with $\tau' < \tau$, so there are no write conflicts. The implementation uses Python's `multiprocessing` with `fork` semantics so that the ∼2.5 GB value table is inherited copy-on-write by worker processes at zero serialization cost.

4.4 Value Storage

Storing one float per reachable state naïvely would require a hash map over ∼30 million keys. Instead, we use dense NumPy arrays shaped to the state-space dimensions:

- Q3/Q4 scrimmage: shape (2, 61, 99, 4, 30, 71), approximately 823 MB. No h dimension because $h = 0$ in the second half.
- Q1/Q2 scrimmage: shape (2, 61, 99, 4, 30, 71, 2), approximately 1.65 GB. The last axis is $h \in \{-1, +1\}$.
- Kickoff and safety-kick arrays: shape (2, 61, 71) or (2, 61, 71, 2), negligible size.

Total memory: approximately 2.47 GB. Unsolved slots are initialized to `NaN`; any read of an unsolved slot raises a `KeyError`, detecting dependency errors immediately.

5 Timeout Decisions

Each team is entitled to three timeouts per half. Calling a timeout after a play reduces the clock charge according to Table 4.
Calling a timeout is therefore strategically useful only after a 3-tick (long gain, ≥ 20 yards) or 2-tick (normal in-bounds gain) play, where it saves 2 or 1 tick respectively.

Table 4: Clock time counted after a timeout is called, by original play duration.

Original play duration	Ticks without TO	Ticks after TO
45 sec (long gain)	3	1
30 sec (normal play)	2	0
15 sec (OOB / penalty)	1	0

5.1 The Post-Play Timeout Sub-Game

After each play completes with tick cost k, either team may call a timeout to reduce the clock charge to $k' < k$. Let

$$\begin{aligned} a &= V^{(\theta_1-1,\,\theta_2)}(s',\,k') \quad \text{(offense calls)},\\ b &= V^{(\theta_1,\,\theta_2-1)}(s',\,k') \quad \text{(defense calls)},\\ c &= V^{(\theta_1,\,\theta_2)}(s',\,k) \quad \text{(neither calls)}. \end{aligned}$$

Because this is a zero-sum game, reduced clock time can benefit at most one team, so both calling simultaneously is never rational. Since at most one team will call, the order of decisions is irrelevant: whichever team benefits simply calls, and the other does not. The sub-game value is therefore

$$V_{\text{TO}} = \max\big(a,\ \min(b,\,c)\big), \tag{2}$$

which replaces the plain $V^{(\theta_1,\theta_2)}(s',k)$ lookup in the outer LP.

5.2 Layered Solve Architecture

Adding timeouts to the state tuple would multiply the state space by $4^2 = 16$. Instead, we maintain 16 separate value tables $V^{(\theta_1,\theta_2)}$, each with the same array layout as the no-timeout base (≈ 823 MB per table, ≈ 13 GB total). The tables are solved in order of increasing $\theta_1 + \theta_2$:

$$(0,0)\ \to\ (1,0),\,(0,1)\ \to\ (2,0),\,(1,1),\,(0,2)\ \to\ \cdots\ \to\ (3,3).$$

The $(0,0)$ table (no timeouts remaining) is solved first and serves as the base case. Each subsequent (θ_1,θ_2) requires a full backward-induction pass, with the post-play sub-game looking up already-solved predecessor tables. The game-relevant output is $V^{(3,3)}$: both teams begin with three timeouts.

6 Comparison to McCulloch's Method

Table 5 summarizes the principal differences between McCulloch's approach and ours.

The most consequential difference is the matrix entry. In McCulloch's formulation, $A^{(s)}_{a,b}$ is an approximate scalar that proxies game value. In ours, it is $Q_s(a,b) = \mathbb{E}[V(s') \mid s,a,b]$, which is the exact expected value of the resulting position under equilibrium play from that position onward. Consequently, our equilibrium mixed strategy is a Nash equilibrium for the *game of Football Strategy* itself, not merely for the local matrix game with surrogate payoffs.

Table 5: McCulloch's heuristic agent vs. the exact backward-induction solver.

Aspect	McCulloch (2004)	This work
Matrix entry value	Heuristic bonus on raw yardage	Exact continuation value $Q_s(a, b)$
Optimality guarantee	Local Nash per play; global not guaranteed	Global Nash for the full game
Clock	Not modeled explicitly	Exact 15-sec tick accounting, two-minute warning, timeouts
Fourth-down decisions	Fixed heuristic cutoffs	Optimal from $V(s)$ for punt/FG/go
Score sensitivity	Bonus scaling by score margin	Terminal utility $U(\delta) \in \{-1, 0, +1\}$
Field goal	Heuristic range threshold	Exact probability by distance
Kickoff / safety kick	Not modeled as strategic decisions	Full stochastic transitions; onside kick option included
Punts	Fixed utility for receiving position	Optimal punt vs. go-for-it trade-off
Timeouts	Not modeled	Pure-strategy sequential sub-game; 16-table layered solve
State space solved	Each play independently	$\sim$30M states $\times$ 16 TO combos
Tunable parameters	Several heuristic constants	None

On the correctness of heuristic strategies. It is in principle possible that McCulloch's heuristics happen to produce strategies close to the true equilibrium; whether this is so is an empirical question that our solver can now answer directly, by comparing the heuristic mixed strategies to the exact equilibrium strategies at each state.

7 Red-Zone Play Restrictions and Chart Parsing

The Ball Control Offense Plays Chart restricts legal play sets near the opponent's goal line. In offense-relative coordinates:

- Plays 17–20 are illegal when $x \geq 80$ (within the opponent's 20).
- Plays 13–16 are illegal when $x \geq 90$ (within the opponent's 10).

The chart is parsed cell-by-cell into a typed intermediate representation before solving. Each cell token is classified as one of: YARDS, INCOMPLETE, FUMBLE, LONGGAIN, INTERCEPTION(r), PENALTY(y), LOSSOFDOWN(y), or an OR-choice CHOICE($\text{branch}_1, \text{branch}_2, \text{resolver}$) where `resolver` $\in \{\text{offense}, \text{defense}\}$ indicates which team selects the better branch. This separation between parsing and solving simplifies both the transition engine and the LP construction.

A subtle rule applies to interceptions with large negative return yards (the intercepting defender runs toward their own end zone): if the resulting field position exceeds the physical boundary of the defender's end zone (more than 110 yards from the offense's own goal line in offense-relative coordinates), the play is treated as an incomplete pass rather than a touchback.

8 Scale and Computational Results

The fourth-quarter solve enumerates approximately 30.9 million reachable states across 60 clock layers via forward BFS. Of these, approximately 615,000 are unique *scrimmage core* states (the (x, d, y, δ, h) component), with each appearing at multiple clock values. Each scrimmage state requires solving a matrix LP with at most 22 rows (offense actions) and 10 columns (defense cards).

The no-timeout base ($\theta_1 = \theta_2 = 0$) runs in approximately 8 hours on an 8-core machine; the full 16-table solve requires approximately 16× that time. Each value table is stored in approximately 823 MB as a dense NumPy array.

The value $V(s_0)$ at the initial kickoff state ($q = 4, \tau = 60, \delta = 0$) gives the exact win probability for the team kicking off at the start of the fourth quarter under a tied score. Any deviation from the equilibrium mixed strategies by either player can only decrease that player's win probability.

References

[1] Sean McCulloch. A game-theoretic intelligent agent for the board game *Football Strategy*. *Unpublished manuscript*, 2004.

[2] John von Neumann. Zur Theorie der Gesellschaftsspiele. *Mathematische Annalen*, 100(1):295–320, 1928.

[3] Lloyd Shapley. Stochastic games. *Proceedings of the National Academy of Sciences*, 39(10):1095–1100, 1953.

[4] Martin Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.

[5] Julian Hall, Ivet Galabova, Leona Gottwald, and Michael Feldmeier. HiGHS: High performance software for linear optimization. https://highs.dev, 2023.